Complete Contracts for EiffelBase

Marco Zietzling
02-906-998
Semester Thesis
Summer 2007

Chair of Software Engineering
Department of Computer Science
ETH Zürich

Bernd Schoeller
Prof. Bertrand Meyer

Abstract

Acknowledgments

I would like to thank my supervisor Bernd Schoeller for his competent support and helpful advices. He always had time for answering my questions and giving hints concerning implementation and general library design. I also would like to thank the other members of the ballet team (Raphael Mack and Jason (Yi) Wei).

Additionally I would like to thank my girlfriend, my family and my friends for the support during my studies and this thesis.

Contents

Abstract **iii**

Acknowledgments **v**

1 Introduction **1**
1.1 Motivation 1
1.2 Related Work 2
1.3 Overview 2

2 Frames **3**
2.1 Frame problem 3
2.2 Dynamic Frames 4
2.3 Dynamic Frames in Eiffel 6

3 Models **7**

4 Library Design **9**
4.1 EiffelBase with Complete Contracts 9
4.2 Differences to EiffelBase 9

5 Conclusions **11**

Bibliography **12**

Chapter 1

Introduction

1.1 Motivation

The contracts currently used in Eiffel (cf. [Mey92b, Mey92a, Mey97]) are strong underspecifications. There are always some properties when invoking Eiffel features that are not expressed through the contracts. This is specially true when it comes to express what does not change through feature invocation. This makes the formal verification of Eiffel impossible, as in the case of verification we have to assume the worst possible implementation that still satisfies the contract.

To overcome these lacks, two approaches will be used to add full functional specifications to Eiffel: Models and Frames.

- Models are mathematical structures which describe the abstract state of an object. These mathematical structures are built out of sets, relations, functions and sequences. The currently available library called MML (cf. [Wid04], downloadable from [mml]) provides this functionality. This model library is based on a typed set theory of finite sets.
- Dynamic Frame Contracts (DFC) (TODO zitat) enable us to talk about infinite sets of objects. They are contractual expressions describing the read effect and the write effect of a feature. With DFC we can solve the "frame problem" and exclude unwanted side effects.

This semester thesis is an effort to redevelop a major part of the EiffelBase Library to support Dynamic Frame Contracts and Models. The goal is to

redesign and implement the linked list class in the EiffelBase Library and all supporting classes to allow full functional specifications and therefore formal verification. The redesign of the EiffelBase Library structure should lead to a cleaner and smoother Base Library.

1.2 Related Work

1.3 Overview

In Chapter 2 we present a short overview over the concept of Frames. In Chapter 3 a short introduction into the concept of Models is given. In Chapter 4 the redesigned and reimplemented Library is presented. In particular the new Library using DFCs and Models is introduced (Section 4.1) and differences to the original EiffelBase Library are emphasized (Section 4.2). Finally we present our results, conclusions and ideas for future work in Chapter 5.

Chapter 2

Frames

In this Chapter the frame problem is introduced and a short overview over the topic of Frames and in particular Dynamic Frames is given. For more information see also [Kas06, SO, Sch06].

2.1 Frame problem

The frame problem, the problem of how to formalize framing requirements, is a central issue in formal logic, first studied in the context of artificial intelligence in the 1960s. It was initially formulated as the problem of expressing a dynamical domain in logic without explicitly specifying which conditions are not affected by an action. Artificial intelligence tries to model the logical thinking of a human being. For the brain it is fairly simple to connect many things logically. But for a computer or a robot, an infinite number of things has to be specified. The frame problem is that specifying only which conditions are changed by an action do not allow, in logic, to conclude that all other conditions are not changed. To solve this problem one could write so called frame axioms which are logic formulas which explicitly describe the non-effects of each action. But since most actions do not affect most properties this solution is not suitable for large systems.

The frame problem has then been taken up by philosophers. In philosophy, it is about rationality in general. Whether it is possible to limit the scope of the reasoning required to derive the consequences on an action. In computer science the problem gained importance concerning program verification. Program verification tools are but logic tools which analyze a pro-

gram. Knowing what the program should do, for example from a contract, and then decide if the program really does things right.

A significant aspect of the behavior of any operation is what parts of the world it leaves unchanged. Usually, a specification for an operation is split into a functional requirement, which describes what changes the specified operation brings about and a framing requirement, which describes the frame of the operation so the part of the world which the operation is allowed to change. Anything outside the frame is left unchanged. In a modular setting we don't know all the variables of the program at the time we specify a computation. So there is no proper way to write we change x but we don't touch y and z. The only way is to say we modify x in a contract of a method and this statement is then translated at the client. So if the client or the implementer of an interface introduces two new variables y and z, the statement we modify x is translated to we modify x and don't touch y and z. But in the presence of pointers, this translation might be unsound. Independence of variables is important when guarantee absence of unwanted aliasing. But the combination of two important features of object oriented programming, namely encapsulation and the support for pointers, makes the frame problem really hard to solve because the combination introduces the possibility of aliasing.

2.2 Dynamic Frames

The frame problem in general is to specify which variables are affected by a computation and which are not. It is important for program verification but difficult to solve when using the common object oriented programming features like encapsulation and pointers. Existing solutions concerning the frame problem guarantee the absence of unwanted aliasing but for the price of more or less strong restrictions for the programmer. [Kas06] introduced a formal theory that supports specification variables and pointers without programming restrictions. The specification language is strong enough to express the desired property that at the present state, the values of two variables are independent.

For Kassios, a frame is nothing more than a specification variable whose value is a set of memory locations. So the new contribution of Kassios is that the frame itself is part of the state. The frame is like a function depending on the state. It is not a constant but it is dynamic which means the set of memory locations can change. For example suppose we have a frame which

frames a whole list of elements. If we insert a new element, it is automatically framed too and its memory address is added to the frame variable. The virtual address of the frame or the specification variable respectively can be itself part of the frame. Which in fact means that frames could frame themselves.

Kassios introduced three new definitions or framing specifications concerning frames: The first specification (Equation 2.1) is preservation which is satisfied by every computation that does not touch region f.

$$\Xi f := \sigma' \triangleright f = \sigma \triangleright f \tag{2.1}$$

The second specification (Equation 2.2) is modification which is satisfied by every computation that only touches region f or at most allocates new memory. So all allocated locations other than f are not touched.

$$\Delta f := \Xi(\text{Used} \setminus f) \tag{2.2}$$

The third specification (Equation 2.3) is the swinging pivot requirement which does not allow the frame f to increase in any other way than allocation of new memory.

$$\Lambda f := f' \subseteq f \cup \text{Unused} \tag{2.3}$$

Equation 2.4 states that a variable v depends only on locations in f, so leaving these locations untouched preserves the value of v.

$$f \textbf{ frames } v := \forall \sigma' \cdot \Xi f \Rightarrow v' = v \tag{2.4}$$

Independence of two variables can be expressed as disjointness of dynamic frames. Disjointness of frames is an important property. Therefore it is desired to establish and preserve disjointness. To preserve disjointness, dynamic frames usually frame themselfes.

$$f \textbf{ frames } x \ \wedge \ g \textbf{ frames } y \ \wedge \ \mathit{disjoint}[f; g] \ \wedge \ \Delta f \Rightarrow y' = y \tag{2.5}$$

$$\mathit{disjoint}[f; g] \ \wedge \ \Delta f \ \wedge \ \Lambda f \Rightarrow \mathit{disjoint}[f'; g'] \tag{2.6}$$

2.3 Dynamic Frames in Eiffel

Chapter 3

Models

Chapter 4

Library Design

4.1 EiffelBase with Complete Contracts

4.2 Differences to EiffelBase

Chapter 5

Conclusions

Bibliography

[Kas06] Ioannis T. Kassios. Dynamic frames: Support for framing, dependencies and sharing without restrictions. *Formal Methods*, 2006.

[Mey92a] Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, October 1992.

[Mey92b] Bertrand Meyer. *Eiffel: the language*. Object-Oriented Series. Prentice Hall, New York, NY, 1992.

[Mey97] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2 edition, 1997.

[mml] *MML*. http://se.inf.ethz.ch/people/schoeller/mml.html.

[Sch06] Bernd Schoeller. Eiffel0: An object-oriented language with dynamic frame contracts. Technical Report 542, Chair of Software Engineering, Department of Computer Science, ETH Zurich, December 2006.

[SO] Bernd Schoeller and Jonathan Ostroff. Dynamic frame contracts. to be published.

[Wid04] Tobias Widmer. Reusable mathematical models. Master's thesis, ETH Zurich, July 2004.